AWS SAA-C03 Study Notes: Auto Scaling Groups (ASG)

1. Auto Scaling Group (ASG) Basics

Goal: Scale EC2 instances out (add) to match increased load and scale in (remove) to save money when load decreases. Ensure a minimum and maximum number of instances run.

- **Min Size:** The ASG will never go below this number of instances.
- **Max Size:** The ASG will never go above this number.
- **Desired Capacity:** The number of instances you want running *right now*.
  - *Note:* If you manually change Desired Capacity, the ASG will immediately launch or terminate instances to match it.

Launch Template vs. Launch Configuration (Critical for Exam)

The exam often asks which one to use. **Always choose Launch Templates.**

Feature	Launch Template (Newer/Recommended)	Launch Configuration (Legacy)
Modifiable?	**Yes** (Versioning supported)	**No** (Immutable - must recreate)
Instance Types	Mixed Instances (Spot + On-Demand)	Single Instance Type only
Features	T2/T3 Unlimited, Dedicated Hosts	Limited feature set
Exam Verdict	**ALWAYS USE THIS**	Only use if explicitly forced by legacy constraints

2. Scaling Policies (Detailed)

The SAA-C03 exam focuses heavily on knowing *which* policy fits a specific scenario.

A. Dynamic Scaling Policies

Reactive scaling based on real-time metrics (e.g., CPU, Network In/Out).

1. Target Tracking Scaling (Most Common)

- **Concept:** "I want to keep the average CPU at 50%."
- **How it works:** You set a target value (e.g., 40% ASG Average CPU). The ASG automatically creates CloudWatch alarms. If CPU > 40%, it adds instances. If CPU < 40%, it removes them.

- **Exam Keyword:** "Maintain average metric," "Easiest to set up."

2. Step Scaling

- **Concept:** "If CPU is slightly high, add 1 instance. If CPU is VERY high, add 5 instances."
- **How it works:** You define steps based on alarm breach size.
    - *Example:* * CPU 50-60% -> Add 1 instance.
        - CPU > 60% -> Add 3 instances.
- **Key Feature:** Does **NOT** wait for the cooldown period during scale-out (reacts faster to spikes than Simple Scaling).

3. Simple Scaling (Legacy)

- **Concept:** "If alarm breaches, add 1 instance."
- **How it works:** Relies on a single CloudWatch alarm.
- **Downside:** It must wait for a **Cooldown Period** (default 300s) to expire before scaling again. Step Scaling is generally preferred.

B. Predictive Scaling

- **Concept:** Uses Machine Learning to look at historical data and predict future traffic.
- **How it works:** Forecasts load for the next 48 hours and schedules scaling actions *in advance*.
- **Exam Scenario:** "Recurring business patterns" or "Scale proactively *before* the load hits."

C. Scheduled Scaling

- **Concept:** Scaling based on a known time schedule.
- **Exam Scenario:**
    - "Every Friday at 5 PM, traffic spikes."
    - "Big marketing event next Monday at 9 AM."

3. Important ASG Mechanics

Scaling Cooldowns

- **Definition:** A "pause" period after a scaling activity (default 300 seconds) to allow metrics to stabilize.
- **Effect:** During a cooldown, the ASG will not launch or terminate additional instances (Simple Scaling respects this; Step Scaling can override it for scale-out).
- **Tip:** If your ASG is scaling up and down too rapidly ("thrashing"), increase the cooldown.

Health Checks (EC2 vs. ELB)

- **EC2 Status Checks (Default):** The ASG checks if the instance VM is running. If the hardware fails, ASG replaces it.
- **ELB Health Checks (Optional but Important):** The ASG checks the *application* health via the Load Balancer.
    - *Scenario:* The instance is running (EC2 check PASS), but the App crashed (HTTP 500 error).
    - *If ELB Check Enabled:* ASG sees the App is down, terminates the instance, and replaces it.
    - *If ELB Check Disabled:* ASG thinks the instance is healthy and keeps sending traffic to a broken app.

Termination Policy

When scaling in (removing instances), which one gets killed first?

1. **Default Policy:**
    1. Find the Availability Zone (AZ) with the *most* instances (to maintain balance).
    2. Delete the instance with the **Oldest Launch Configuration/Template**.
    3. If multiple exist, delete the one closest to the **next billing hour**.

- **Exam Tip:** You can enable **Instance Scale-In Protection** to prevent specific instances from being terminated during scale-in (useful for long-running jobs).

4. Advanced Features (High Exam Probability)

Lifecycle Hooks

Allows you to pause the creation or termination of an instance to perform a custom action.

- **Pending:Wait:** Instance is created but paused before serving traffic. (Use case: Installing software, downloading large datasets).
- **Terminating:Wait:** Instance is paused before destruction. (Use case: Uploading logs to S3, extracting final data).
- **Integration:** Usually triggers EventBridge or SNS -> Lambda Function.

Instance Refresh

- **Goal:** Update all instances in the ASG to a new Launch Template (e.g., new AMI or User Data) without downtime.
- **Mechanism:** Performs a "Rolling Update." It replaces instances batch-by-batch.
- **Checkpoints:** You can set percentages (e.g., "replace 20%, then wait 1 hour") to verify stability.

Warm Pools

- **Problem:** Instances take too long to boot and install software (e.g., 5+ minutes), making dynamic scaling too slow.
- **Solution:** A pool of pre-initialized EC2 instances kept in a `Stopped` state.
- **Benefit:** When the ASG scales out, it pulls from the Warm Pool (starting a stopped instance is much faster than creating a new one).

5. Summary Cheat Sheet for Scenarios

Scenario	Solution
Traffic is completely unpredictable/spiky	**Target Tracking** (set CPU/Network metric).
Traffic has a known weekly pattern	**Scheduled Scaling**.
Need to scale BEFORE the load hits (based on history)	**Predictive Scaling**.
Need to prevent a specific instance from terminating	Enable **Scale-In Protection**.
App takes 10 mins to install on boot	Use **Warm Pools** (stopped state) or **Golden AMI**.
Need to run a script *before* instance terminates	**Lifecycle Hook** (`Terminating:Wait`).
Update OS patches on all ASG instances	**Instance Refresh** (Rolling update).
Availability Zones are unbalanced	ASG will attempt to **Rebalance** (launch in one AZ, terminate in another).